

CineMatch AI

AI-Powered Movie Recommendation System

A Machine Learning Based Recommendation Engine

Using Content-Based Filtering and Streamlit

TECHNICAL PROJECT REPORT

Prepared By

Karishma

Bachelor of Technology (B.Tech)

Artificial Intelligence and Data Science

Guru Jambheshwar University of Science

and Technology (GJUST)

Hisar, Haryana

Project Domain

Machine Learning • Recommendation Systems

Artificial Intelligence • Data Analytics

Web Application Development

Academic Session 2026–2027

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the development of this project titled “**CineMatch AI – AI-Powered Movie Recommendation System.**” This project has been an excellent learning experience that enhanced my understanding of Machine Learning, Data Analysis, and Web Application Development.

I am grateful to my family and friends for their constant motivation, encouragement, and moral support throughout the project development process.

I would also like to acknowledge the developers and contributors of various open-source libraries and technologies such as **Python, Streamlit, Pandas, Scikit-learn, Plotly**, and the public APIs including **OMDb API** and **YouTube Data API**, which played an essential role in building this application.

Finally, I sincerely thank everyone who directly or indirectly contributed to the successful completion of this project.

ABSTRACT

Movie recommendation systems help users discover movies that match their interests from a large collection of available content. This project presents **CineMatch AI**, an AI-powered movie recommendation system developed using **Python, Machine Learning**, and **Streamlit**.

The system uses a **Content-Based Filtering** approach to recommend similar movies by analyzing metadata such as genres, cast, keywords, and descriptions. **CountVectorizer** and **Cosine Similarity** are used to calculate similarities between movies and generate personalized recommendations.

The application provides features such as movie search, AI-based recommendations, movie posters, official trailers, favorites management, and an interactive analytics dashboard. It also integrates **OMDb API** and **YouTube Data API** to enhance the user experience with additional movie information.

This project demonstrates the practical implementation of Machine Learning, data preprocessing, API integration, and web application development while providing an efficient and user-friendly movie recommendation platform.

Keywords: Movie Recommendation System, Machine Learning, Content-Based Filtering, Cosine Similarity, Streamlit, Python.

TABLE OF CONTENTS

<i>Acknowledgement</i>	2
<i>Abstract</i>	2
<i>Table of Contents</i>	3
<i>CHAPTER 1 INTRODUCTION</i>	5
<i>1.1 Introduction</i>	5
<i>1.2 Problem Statement</i>	5
<i>1.3 Objectives</i>	5
<i>1.4 Scope of the Project</i>	6
<i>CHAPTER 2 LITERATURE REVIEW</i>	7
<i>2.1 Overview</i>	7
<i>2.2 Existing Recommendation Systems</i>	7
<i>2.3 Research Gap</i>	8
<i>CHAPTER 3 DATASET AND TECHNOLOGY STACK</i>	9
<i>3.1 Dataset Description</i>	9
<i>3.2 Technology Stack</i>	9
<i>3.3 APIs Used</i>	10
<i>3.4 Development Tools</i>	10
<i>CHAPTER 4 METHODOLOGY</i>	11
<i>4.1 System Workflow</i>	11
<i>4.2 Data Preprocessing</i>	11
<i>4.3 Feature Extraction</i>	11
<i>4.4 Similarity Calculation</i>	11

4.5 Web Application Development	11
4.6 Workflow Diagram	12
CHAPTER 5 SYSTEM DESIGN AND IMPLEMENTATION	13
5.1 System Architecture	13
5.2 Project Modules	13
5.3 Project Structure	13
5.4 User Interface	15
CHAPTER 6 RESULTS AND DISCUSSION	16
6.1 Experimental Results	16
6.2 System Performance	16
6.3 Challenges Faced	16
6.4 Future Enhancements	17
CHAPTER 7 CONCLUSION	18
7.1 Conclusion	18
References	18

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid growth of online streaming platforms has significantly increased the availability of digital entertainment content. With thousands of movies available across different genres and languages, users often find it difficult to discover movies that match their interests. Intelligent recommendation systems solve this problem by analyzing movie information and suggesting relevant content based on similarity.

CineMatch AI is an AI-powered Movie Recommendation System developed using **Python**, **Machine Learning**, and **Streamlit**. The system uses a **Content-Based Filtering** approach to recommend movies based on their metadata, including genres, cast, crew, keywords, and descriptions. Movie features are converted into numerical vectors using **CountVectorizer**, and similarities between movies are calculated using the **Cosine Similarity** algorithm.

The application provides an interactive dashboard where users can search for movies, receive personalized recommendations, watch official trailers, view movie posters, manage favorite movies, and analyze movie statistics through visual dashboards. The project demonstrates the practical implementation of Machine Learning techniques, API integration, and web application development while addressing a real-world recommendation problem.

1.2 Problem Statement

Modern movie streaming platforms contain an enormous collection of content, making it difficult for users to identify movies that match their preferences. Traditional search methods often rely on manual filtering or popularity-based rankings, which may not provide personalized recommendations.

The objective of this project is to develop an intelligent recommendation system capable of suggesting similar movies based on their content, thereby improving user experience and reducing the time required to discover relevant movies.

1.3 Objectives

The primary objectives of the project are:

- To develop an AI-powered movie recommendation system using Machine Learning.
- To implement a Content-Based Filtering recommendation algorithm.
- To recommend similar movies using CountVectorizer and Cosine Similarity.
- To develop an interactive web application using Streamlit.
- To integrate external APIs for movie posters, trailers, and detailed movie information.
- To provide users with an intuitive dashboard for searching, exploring, and analyzing movies.

1.4 Scope of the Project

The scope of CineMatch AI includes the development of an intelligent recommendation system that suggests movies based on content similarity. The application enables users to search movies, receive personalized recommendations, view movie details, access official trailers, and analyze movie trends through interactive visualizations.

The project focuses on Content-Based Recommendation techniques and serves as a practical demonstration of Machine Learning, Data Processing, API Integration, and Web Application Development. Future enhancements may include collaborative filtering, user authentication, personalized user profiles, and cloud-based database integration.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

Recommendation systems have become an essential part of modern digital platforms. They help users discover relevant content by analyzing available information and predicting user preferences. These systems are widely used in movie streaming platforms, e-commerce websites, music applications, online learning platforms, and social media.

The primary objective of a recommendation system is to reduce information overload and improve user experience by providing personalized suggestions.

2.2 Existing Recommendation Techniques

Content-Based Filtering

Content-Based Filtering recommends items based on their characteristics. In movie recommendation systems, metadata such as genres, cast, directors, keywords, and movie descriptions are analyzed to identify similar movies. This approach does not require user ratings and performs well even when user interaction data is limited.

Collaborative Filtering

Collaborative Filtering recommends movies by analyzing the preferences of similar users. It identifies users with similar viewing patterns and suggests movies that they have enjoyed. Although this approach often produces highly personalized recommendations, it suffers from the cold-start problem when new users or movies have insufficient interaction data.

Hybrid Recommendation Systems

Hybrid Recommendation Systems combine Content-Based Filtering and Collaborative Filtering to improve recommendation accuracy. By utilizing both movie attributes and user behavior,

hybrid systems reduce the limitations of individual approaches and are widely adopted by commercial streaming platforms.

2.3 Existing Platforms

Popular streaming services such as **Netflix**, **Amazon Prime Video**, and **Disney+** employ advanced recommendation systems to enhance user engagement. These platforms use Artificial Intelligence and Machine Learning techniques to analyze viewing history, ratings, search behavior, and movie metadata to generate personalized recommendations.

Inspired by these real-world applications, **CineMatch AI** implements a Content-Based Recommendation approach that recommends movies based on similarity in movie metadata. This approach provides relevant recommendations while maintaining simplicity, computational efficiency, and ease of implementation.

CHAPTER 3

DATASET AND TECHNOLOGY STACK

3.1 Dataset Description

The movie recommendation system uses a movie dataset containing information about thousands of movies. The dataset includes important metadata such as movie titles, genres, cast members, directors, keywords, ratings, popularity, release year, and overview. These attributes are used to generate meaningful recommendations by measuring the similarity between movies.

Before training the recommendation model, the dataset undergoes preprocessing to remove missing values, merge important textual features, and create a single **tags** column. This processed data serves as the input for the recommendation engine.

3.2 Technology Stack

The project was developed using modern technologies and open-source tools.

Technology	Purpose
Python	Core programming language
Streamlit	Interactive web application development
Pandas	Data loading and preprocessing
NumPy	Numerical computations
Scikit-learn	Machine Learning algorithms
Plotly	Interactive data visualization
Requests	API communication
Git & GitHub	Version control and project management

3.3 APIs Used

To improve user experience, external APIs were integrated into the application.

- **OMDb API** – Retrieves movie details such as ratings, release year, runtime, and plot summary.
- **TMDb API** – Fetches high-quality movie posters.
- **YouTube Data API** – Provides official movie trailers directly within the application.

3.4 Development Tools

The project was developed using the following software tools:

- Visual Studio Code (VS Code)
- Jupyter Notebook
- Git
- GitHub
- Streamlit Community Cloud

CHAPTER 4

METHODOLOGY

4.1 System Workflow

The development of CineMatch AI follows a structured workflow consisting of data collection, preprocessing, feature engineering, recommendation generation, and user interaction. The movie dataset is first cleaned and transformed into a suitable format before applying Machine Learning algorithms. The generated recommendation model is then integrated into a Streamlit-based web application to provide users with an interactive recommendation experience.

4.2 Data Preprocessing

Data preprocessing is an important step in improving the quality of recommendations. The dataset is cleaned by removing duplicate records, handling missing values, and selecting relevant movie attributes. Important textual features such as genres, keywords, cast members, directors, and movie overviews are combined into a single **tags** column. All text is converted into lowercase and processed into a consistent format for feature extraction.

4.3 Feature Extraction

To represent movie information numerically, the combined **tags** column is transformed into vectors using the **CountVectorizer** technique provided by Scikit-learn. This converts textual movie information into a matrix of word frequencies, enabling mathematical comparison between movies.

4.4 Similarity Calculation

The recommendation engine calculates the similarity between movies using the **Cosine Similarity** algorithm. Movies with higher cosine similarity scores are considered more closely related. When a user selects a movie, the system identifies the five most similar movies and returns them as personalized recommendations.

4.5 Web Application Development

The recommendation model is integrated into a Streamlit web application that provides an interactive user interface. Users can search for movies, view recommended movies, access official trailers, manage favorite movies, and explore movie analytics. External APIs are used to display movie posters and detailed movie information, improving the overall user experience.

4.6 Workflow Diagram

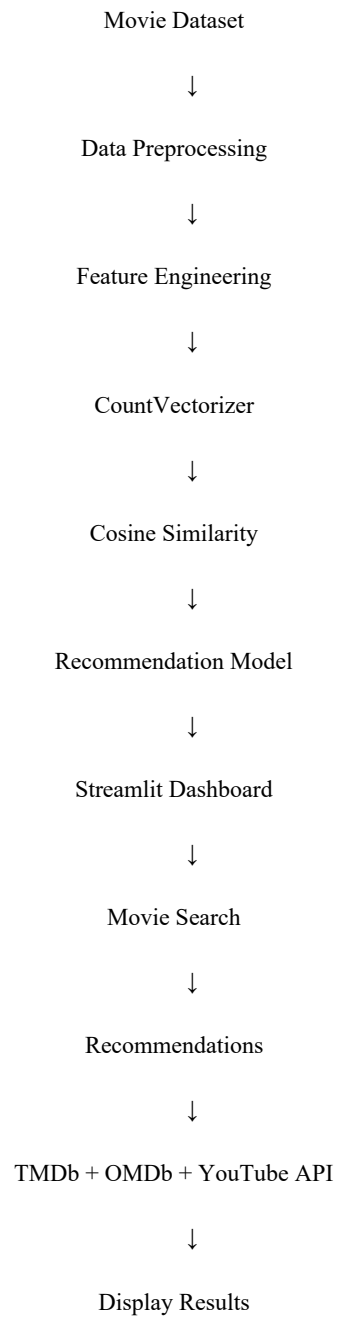


Figure 4.1: Workflow of CineMatch AI Movie Recommendation System

CHAPTER 5

SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Architecture

CineMatch AI follows a modular architecture where each component performs a specific task. The recommendation engine is responsible for generating personalized movie suggestions, while separate modules handle movie details, poster retrieval, trailer retrieval, analytics, and user interface components. This modular design improves code readability, maintainability, and scalability.

The application begins by loading the pre-trained recommendation model and movie dataset. When a user searches for a movie, the recommendation engine identifies similar movies using the cosine similarity matrix. Additional movie information, posters, and trailers are retrieved through external APIs before displaying the final recommendations on the Streamlit dashboard.

5.2 Project Modules

The project consists of the following major modules:

- **Home Module** – Provides the main interface for movie search and recommendations.
- **Recommendation Engine** – Generates personalized movie recommendations using cosine similarity.
- **Movie Details Module** – Displays detailed movie information such as ratings, genres, runtime, and release year.
- **Poster Module** – Retrieves movie posters using the TMDb API.
- **Trailer Module** – Displays official trailers using the YouTube Data API.
- **Analytics Module** – Visualizes movie statistics using interactive Plotly charts.
- **Favourite Module** – Allows users to save and manage their favourite movies.

5.3 Project Structure

The project is organized into separate directories to improve maintainability.

Movie_Recommendation_System

|

|— assets/

|— dashboard/

```
| |— Home.py
| |— recommender.py
| |— analytics.py
| |— details.py
| |— poster.py
| |— youtube_api.py
| |— pages/
|
|— data/
| |— raw/
| |— processed/
|
|— models/
| |— movies.pkl
| |— similarity.pkl
|
|— notebooks/
|— requirements.txt
|— README.md
```

5.4 User Interface

The user interface is developed using **Streamlit** and follows a clean, Netflix-inspired design. The dashboard enables users to search movies, receive recommendations, explore analytics, manage favourites, and view movie details through an interactive and responsive interface.

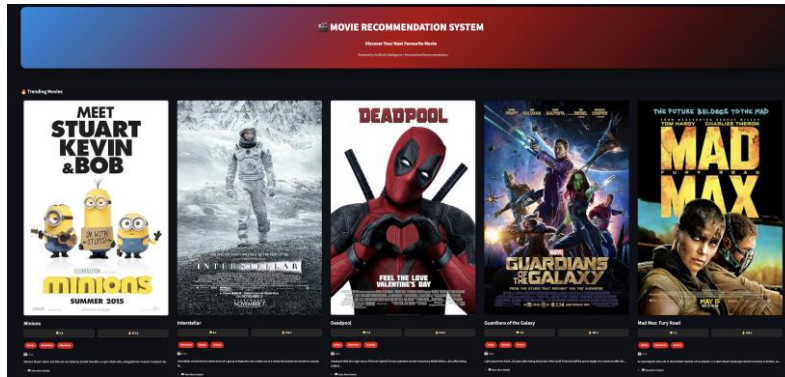


Figure 5.1: Home Dashboard

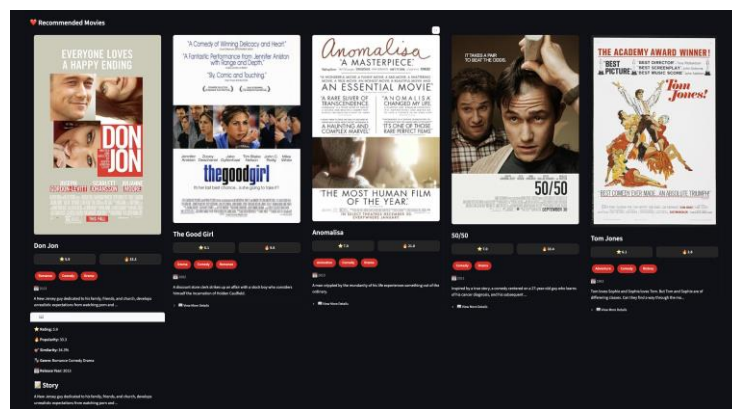


Figure 5.2: Movie Recommendation Interface

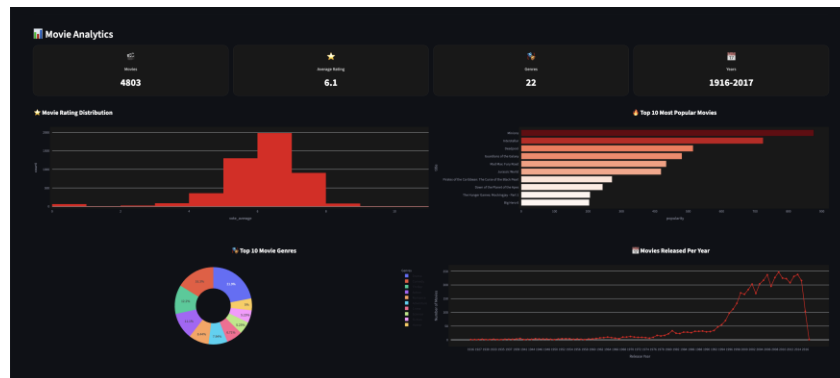


Figure 5.3: Analytics Dashboard

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Experimental Results

The developed CineMatch AI system was successfully implemented and tested using the processed movie dataset. The recommendation engine accurately generates movie suggestions based on content similarity, providing users with relevant recommendations in real time. The integration of external APIs enables the application to display movie posters, detailed information, and official trailers, enhancing the overall user experience.

The Streamlit dashboard provides a responsive and interactive interface where users can search for movies, explore recommendations, manage favourite movies, and visualize movie statistics through interactive charts.

6.2 System Performance

The recommendation system performs efficiently by utilizing pre-trained machine learning models stored as serialized **pickle (.pkl)** files. This approach minimizes computation during runtime and enables faster recommendation generation.

The application demonstrates:

- Fast movie search and recommendation generation.
- Accurate content-based recommendations.
- Smooth integration with external APIs.
- Interactive and user-friendly dashboard.
- Modular architecture for easy maintenance and future development.

6.3 Challenges Faced

During the development of the project, several technical challenges were encountered:

- Cleaning and preprocessing a large movie dataset.
- Optimizing the recommendation model for faster execution.
- Managing API requests and handling missing poster or trailer information.
- Reducing application loading time by implementing Streamlit caching techniques.
- Organizing the project into reusable and maintainable modules.

These challenges were addressed through efficient preprocessing, optimized model loading, caching mechanisms, and modular code organization.

6.4 Future Enhancements

The current system can be further improved by incorporating the following features:

- User authentication and personalized profiles.
- Collaborative and Hybrid Recommendation techniques.
- User rating and review system.
- Watchlist management.
- Cloud database integration.
- Mobile application support.
- Real-time recommendation updates using user interaction history.

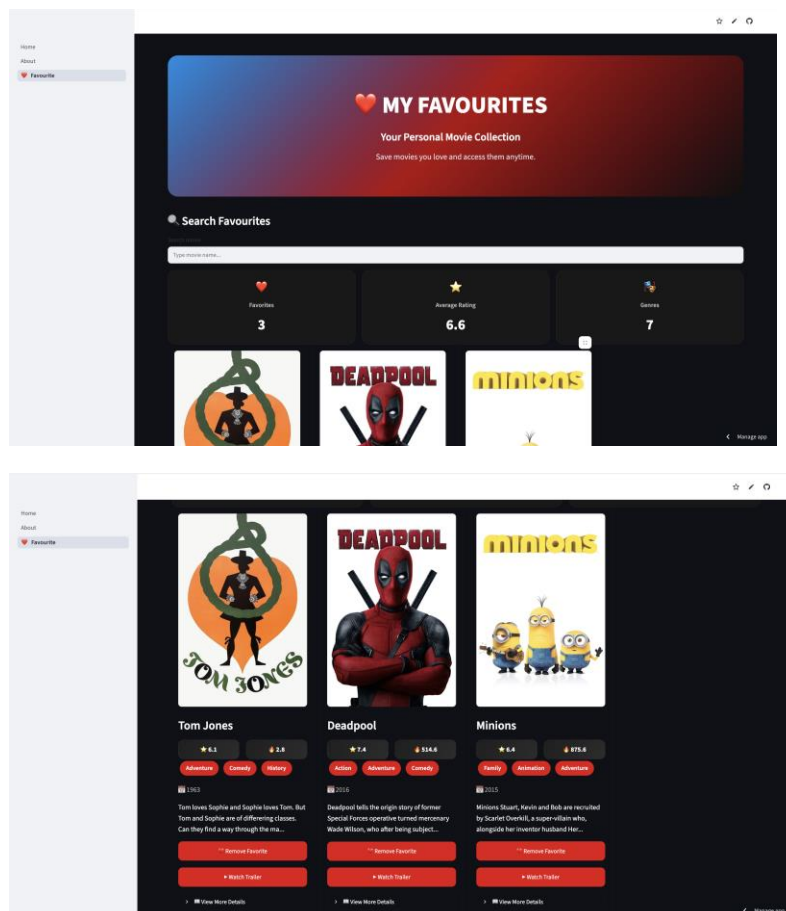


Figure 6.3: Favourite Movies Interface

CHAPTER 7

CONCLUSION

7.1 Conclusion

CineMatch AI successfully demonstrates the application of Machine Learning techniques in developing an intelligent movie recommendation system. By implementing Content-Based Filtering with CountVectorizer and Cosine Similarity, the system provides relevant and personalized movie recommendations through an interactive Streamlit dashboard.

The project integrates external APIs to enhance user experience by displaying movie posters, detailed information, and official trailers. Its modular architecture, efficient recommendation engine, and user-friendly interface make it a practical solution for personalized movie discovery.

Overall, the project strengthened practical skills in Machine Learning, Data Preprocessing, API Integration, Web Application Development, and Version Control using Git and GitHub. It also provides a strong foundation for future enhancements and serves as an effective portfolio project for placements and further academic work.

REFERENCES

1. Pedregosa, F., et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 2011.
2. Pandas Documentation. <https://pandas.pydata.org>
3. Streamlit Documentation. <https://docs.streamlit.io>
4. Plotly Documentation. <https://plotly.com/python>
5. TMDb API Documentation. <https://developer.themoviedb.org>
6. OMDb API Documentation. <https://www.omdbapi.com>
7. YouTube Data API Documentation. <https://developers.google.com/youtube>